

Architectural Patterns

System Design Crash Course: Quick Revision Before Interviews

Rahul Rajat Singh



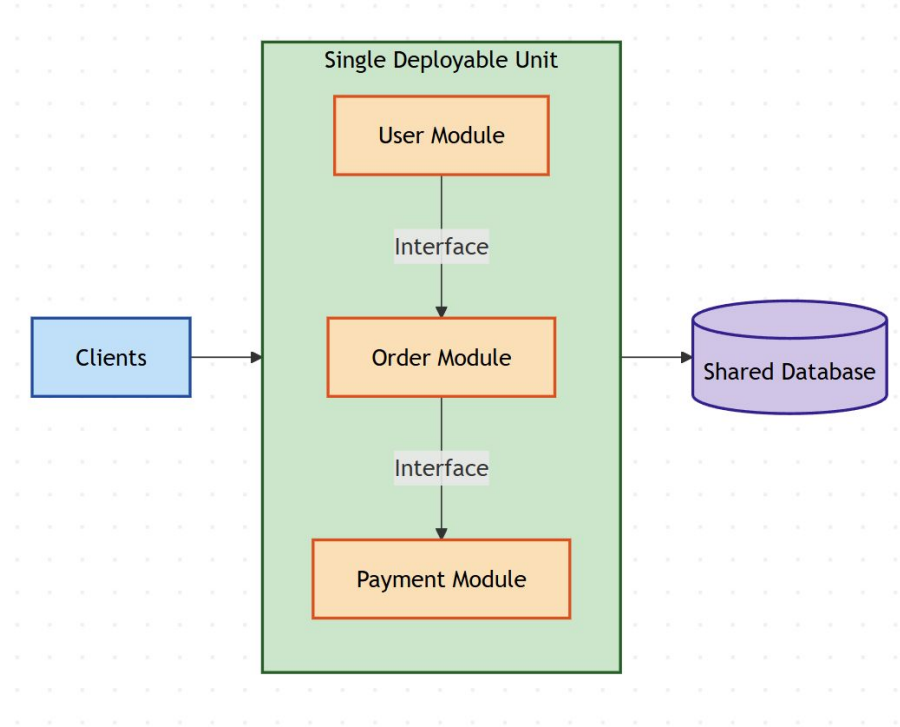
Monolithic Architecture

- Single deployable application
- All components tightly integrated
- Simple development and deployment
- Easy local debugging
- Scaling requires full replication



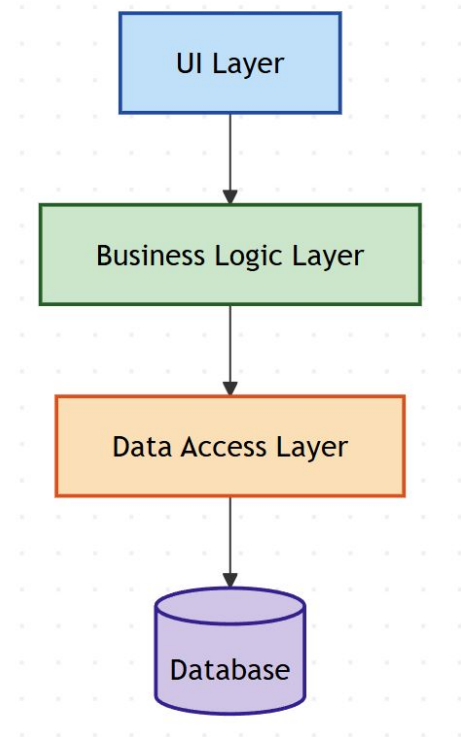
Modular Monolith Architecture

- Single deployable application
- Strong internal module boundaries
- Modules communicate via interfaces
- Avoids distributed system overhead
- Prepares for microservices evolution



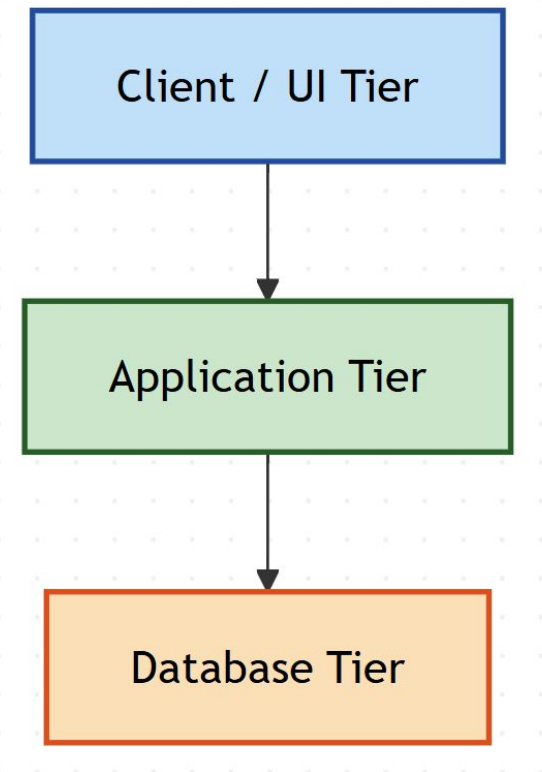
Layered Architecture

- UI, business, data layers
- Each layer has clear responsibility
- Changes isolated per layer
- Improves maintainability and clarity
- Can hide performance issues



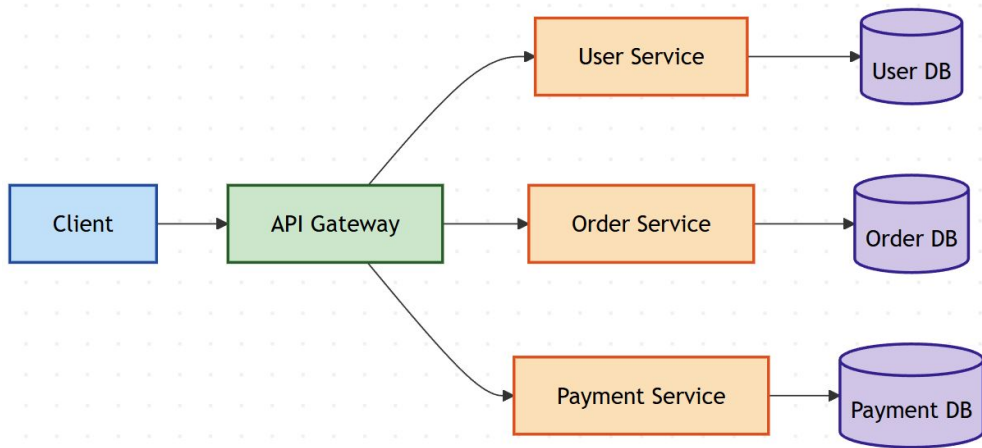
Multi-Tier Architecture

- Separates system into deployment tiers
- Common: 2-tier, 3-tier models
- Tiers deployed independently
- Improves isolation and scalability
- Adds network latency between tiers



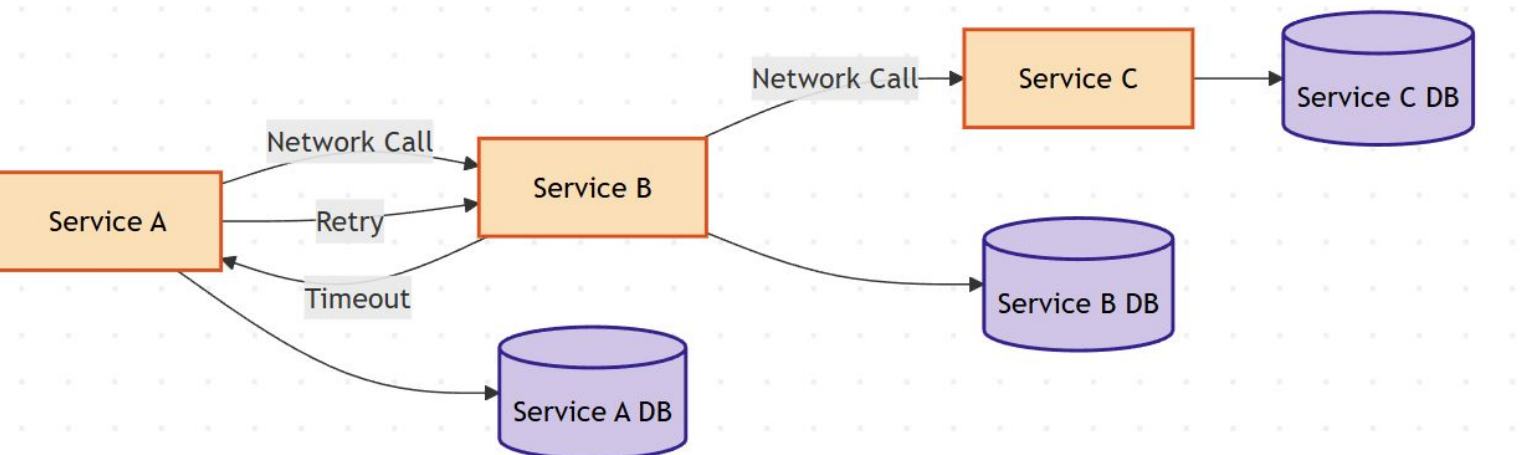
Microservices Architecture

- Independent deployable services
- Clear service boundaries
- Own data per service
- Enables independent scaling
- High operational complexity



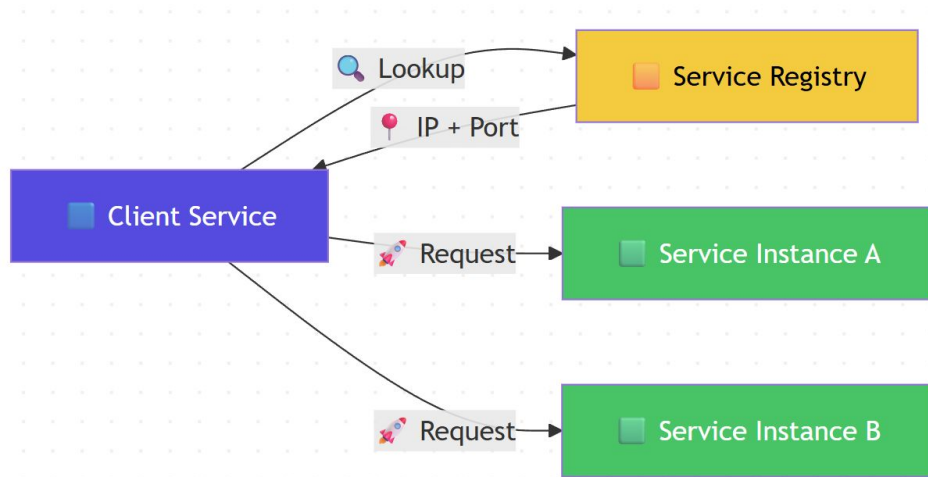
Complexity of Microservices

- Network calls replace method calls
- Data consistency becomes harder
- Operational overhead increases
- Requires strong observability
- Pays distributed systems tax



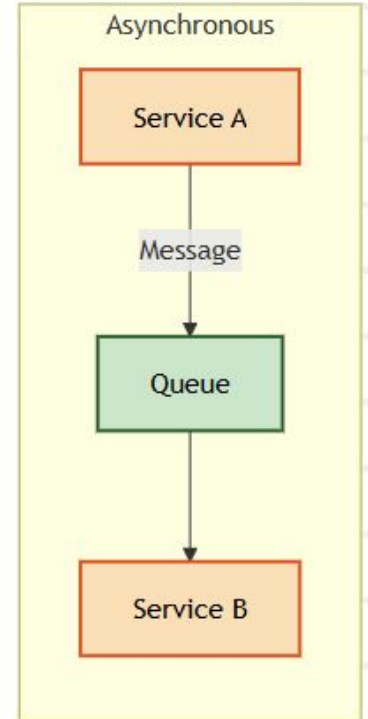
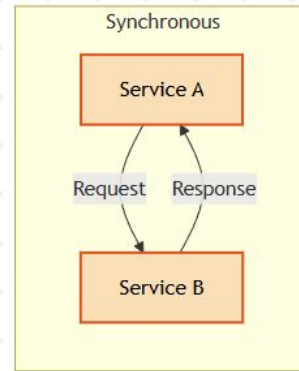
Service Discovery

- Dynamic service location lookup
- No hardcoded IPs or ports
- Enables scaling and failover
- Central registry or DNS-based
- Adds lookup and dependency cost



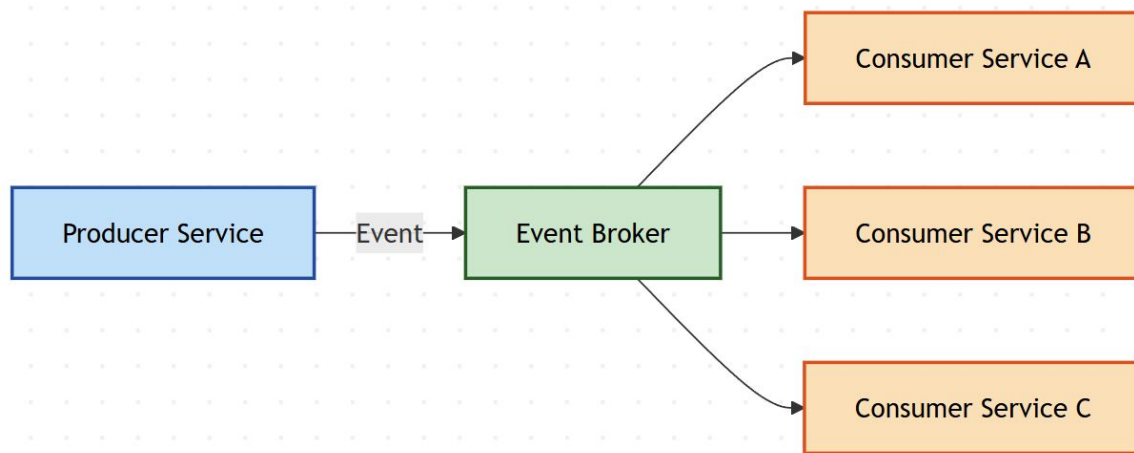
Synchronous vs Asynchronous Communication

- Synchronous: request waits for response
- Asynchronous: fire-and-forget processing
- Sync favors simplicity
- Async improves resilience
- Latency versus reliability trade-off



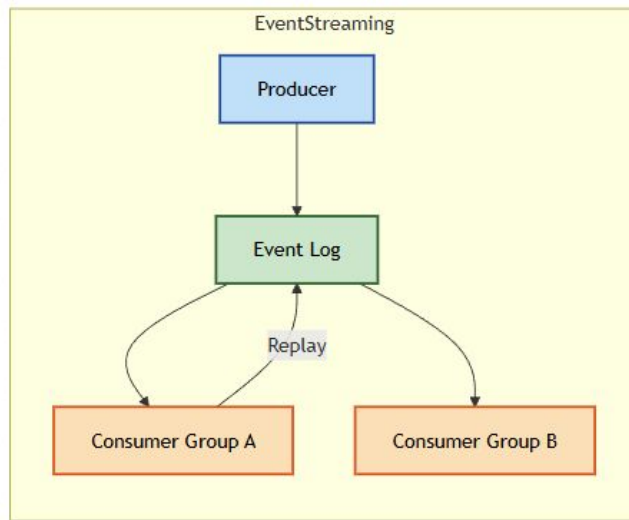
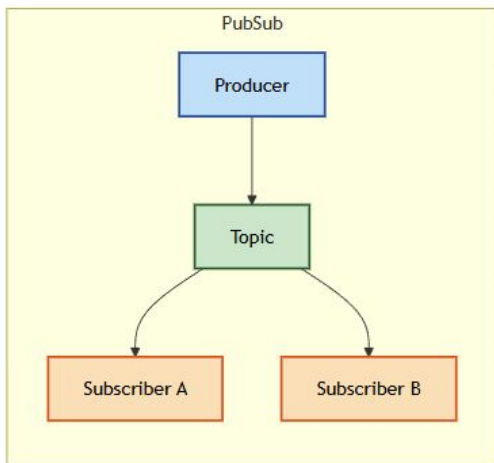
Event-Driven Architecture

- Producers emit events
- Consumers react to events
- No direct service coupling
- Enables asynchronous workflows
- Trades consistency for scalability



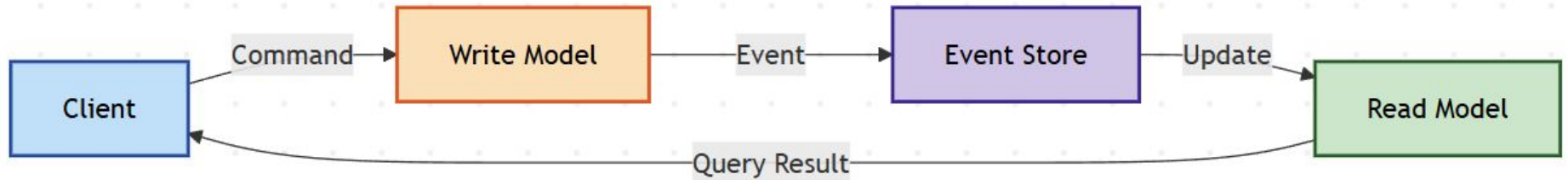
Pub-Sub vs Event Streaming

- Pub-Sub: push-based fan-out
- Event Streaming: durable event log
- Pub-Sub drops past messages
- Streaming supports replay
- Simplicity versus control trade-off



CQRS

- Separates read and write models
- Independent scaling of reads
- Different data representations
- Often paired with events
- Increases system complexity



Eventual Consistency

- Data converges over time
- Temporary inconsistencies allowed
- Common in distributed systems
- Improves availability and scale
- Trades immediacy for resilience

